# Poster explainer: what to say, panel by panel

For a listener with no background. Read the plain text aloud; the grey lines tell you where to point. Every number here is on the poster.

## If you only have 20 seconds

“This poster is about a computer program that solves picture puzzles. When it fails, we want it to work out *why* it failed before it tries to fix itself, because different failures need different fixes, and the wrong fix can break things that used to work. We found a case where the program could have had the answer but never looked at the puzzle the right way. We also found that the obvious fix made other puzzles worse. So we have not solved new puzzles yet. What we have learned is how to diagnose a failure properly first.”

## 1. What are the puzzles?

*[Point to: the small coloured grids in the middle column, “demo 0” to “demo 3”]*

“These are ARC puzzles. Each one is a small grid of coloured squares. The puzzle gives a few examples: an input picture, and the output picture it turns into. The solver has to work out the rule from those few examples, and then use the rule on a new picture.

Look at these four examples. Two yellow dots become a filled yellow square, and the dots mark its corners. So the rule is: draw the box whose corners are the dots, in the dots’ colour. People find puzzles like this easy. Computers find them hard, because every puzzle has a new rule and only a few examples.”

## 2. The question

*[Point to: “The question”, top left]*

“Our question is: how can a program tell that its way of solving is not good enough, and get a better one, without cheating on the test that judges it?

When the program fails, there can be different reasons:

- it did not look hard enough (*search*);
- it looked at the picture the wrong way (*representation*);
- it does not have the tool it needs (*capability*);
- the examples are not enough to pin down the rule (*evidence*);
- a correct answer was blocked by our safety checks (*certification*).

Each reason needs a different fix. It is like a doctor: you diagnose before you prescribe.”

## 3. The CORA loop

*[Point to: the diagram “The CORA loop”, left column]*

“CORA is the name of our system. It works in a loop. The solver tries the puzzle. If it fails, we look at a record of what it did. We decide what kind of failure it was, choose a fix, and test the fix with checks we wrote down in advance. Only after that is the fix allowed to be used.

The solid boxes are built and measured. The dashed boxes are ideas we have not built yet. One honest point: in this work, the step that decides what kind of failure it was was done by hand. It is not automatic yet.”

## 4. Four separate questions

*[Point to: "Four claims, four kinds of evidence", bottom left]*

"We keep four questions apart:

- Did the new tool really cause the success?
- Did the record of the failure matter?
- Is the new tool truly new?
- Does it help later, on other puzzles?

The first two have only early evidence, on practice puzzles. The third is not established. The fourth is not tested yet."

## 5. What we measured

*[Point to: the first bar chart, "27 / 30" and "12 / 30"]*

"First we tested on made-up practice puzzles that we generate ourselves, where we know the right answers. The improved version passed 27 of 30. The earlier version passed 12 of 30. That tells us about made-up puzzles only, not real ones."

*[Point to: the second bar chart, the one that shrinks from 60 to 0]*

"Then we used 60 real ARC practice puzzles. The built-in solver failed 54 of them. In 31 it left useful clues about why. In 4, our system built a new tool from those clues and plugged it in. In 1, the solver accepted the new tool and used it.

But that one success did not survive our strictest check, called 'leave-one-out'. We hide one example, rebuild everything from the others, and see whether the hidden example still comes out right. Think of a student: if you hide one worked example and they can still solve it from the rest, they understood; if not, they may have just memorized. Our system passed 0 of 14 of those checks.

So no new real puzzle was solved, and our official score stays at 185 out of 1,000."

## 6. Why it failed: the right view was pushed out

*[Point to: the big blue-and-orange grid in the middle column]*

"Then we asked why. The program can look at a picture in seven different ways. We call them views, S1 to S7. For example, one view says 'each separate blob is an object'. Another says 'all squares of the same colour together are one object'. The program only has room to search four views, in a fixed order.

With all the examples, only a couple of views pass a basic test, the right one among them, and the program finds the right rule. It is marked G. But when we hide examples and leave only two, more views pass the test by coincidence and fill up the four places. The right view, S6, gets pushed out, so the program never even looks at the right answer. Those are the orange boxes marked 'no G'.

And two of the four places were wasted on views that look exactly the same to the program. It is like a library with four reading desks: if two desks are taken by copies of the same wrong book, the right book never gets read."

## 7. Testing the obvious fix first

*[Point to: the grid with the red boxes, top right]*

"The obvious fix is: force the program to look at a particular view first. Before trying that on real puzzles, we tried it on 24 made-up puzzles where we know the answers. Each row is one puzzle. Grey means nothing changed; red means it got worse.

It helped none of them, and it hurt five. Four that were right became wrong or rejected, and one wrong answer got accepted as if it were right. So we did not use that fix. The lesson: a fix that makes your favourite example work can quietly break others."

## 8. The rules any fix must follow

*[Point to: "What a valid repair must respect"]*

"Any fix must play fair. When we hide an example to test the program, the program is not allowed to peek at it, not even indirectly.

- One idea we had would have peeked indirectly. So we only use it to learn, never as proof.
- Another idea, removing the duplicate views, plays fair. But it would change what the program looks at in about half of the situations we checked. So it must be tested carefully first. We have not run it.
- We also found that hiding an example can sometimes make a view fail the basic test, 7 times out of 724. So we call that an observation, not a rule.

"

## 9. Could this work outside puzzles?

*[Point to: "A portable controller?", marked BIOLOGICAL APPLICATION: UNTESTED]*

"We think the decision-making part could work in other fields: diagnose the failure, choose a fix, check the fix. One example would be predicting how cells respond to a drug or a gene change. The puzzle-specific parts would not carry over. This is only an idea. We have no biology data and no biology results."

## 10. What we are not claiming

*[Point to: "Not claimed", bottom right]*

"To be clear:

- We did not improve the ARC score.
- We did not show that the computer invents truly new ideas.
- We have not built the automatic diagnosis yet.
- A larger test on 120 new puzzles is written down and frozen, but it has not been run.

"

## 11. The one-sentence message

"Before a reasoning system changes itself, it should find out what kind of failure it had, and every fix should be tested fairly, including on cases where the fix could do harm."

## If they ask

**"Did it solve any new puzzles?"** "Not yet. No new real puzzles."

**"Why not just let it search longer?"** "Searching longer would not help here. The right view was never on the list it searched."

**"Why use made-up puzzles?"** "Because there we know the right answers, so we can see when a fix does harm."

**“What happens next?”** “We test the ‘remove duplicates’ fix fairly, with safety checks, before any new real test.”

## Words you may need

| | |
|---|---|
| **ARC** | a public set of picture puzzles used to test reasoning. |
| **example pair** | one input picture and the output it should become. |
| **rule** | what turns the input into the output. |
| **solver, engine** | the program that looks for the rule. |
| **view** | one way of cutting a picture into objects. |
| **leave-one-out** | hide one example and check that the rest still give it correctly. |
| **made-up (synthetic)** | puzzles we generate ourselves, with known answers. |
| **false acceptance** | a wrong answer that passed the checks. |